Kernel Mode e User Mode

Una caratteristica fondamentale di tutti i sistemi operativi moderni è la separazione delle istruzioni più critiche, che la CPU può svolgere solo in un particolare stato, detto **Kernel Mode** (Supervisor Mode, Priviliged Mode o System Mode), e quelle che possono tranquillamente essere svolte in modalità **User Mode**, in quanto non critiche.

Sostanzialmente, quando la CPU lavora in Kernel Mode può eseguire qualsiasi tipo di istruzione e ha accesso a qualsiasi risorsa hardware, mentre quando lavora in User Mode presenta notevoli limitazioni, che servono per evitare che i programmi utente richiedano l'esecuzione di istruzioni privilegiate, interferiscano con altri programmi o addirittura con il Sistema Operativo stesso.

A livello di CPU, è presente un bit il cui valore indica in quale delle due modalità sta attualmente lavorando (tipicamente il valore 0 indica la Kernel Mode, il valore 1 la User Mode). Quando l'elaboratore viene acceso, la CPU parte in Kernel Mode e carica in memoria RAM il Sistema Operativo; dopodiché passa in User Mode ed è pronta per eseguire i programmi utente.

Dal momento che la modalità di esecuzione della CPU dipende dal valore di un certo bit, ovvero viene gestita a livello hardware, questo meccanismo non è aggirabile in alcun modo, in quando solamente il S.O. è in grado di modificare il valore di tale bit per cambiare la modalità di esecuzione della CPU.

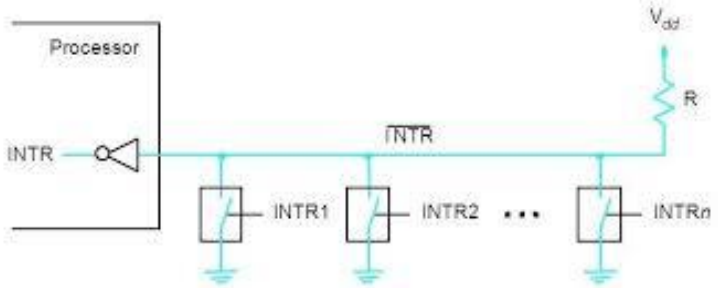
Se un normale programma utente provasse a richiedere l'esecuzione di una istruzione privilegiata, verrebbe bloccato tramite un interrupt hardware generata durante il controllo del valore del bit che indica la modalità di esecuzione della CPU.

Ma come avviene il passaggio da User Mode a Kernel Mode o viceversa?

Una prima modalità con la quale avviene questo passaggio è automatica: ogni periferica è in grado di segnalare degli eventi alla CPU generando un **interrupt hardware**, ovvero inviando un segnale su una linea dedicata del bus di controllo (control bus) della CPU. Per esempio, la tastiera può generare un interrupt ogni volta che viene premuto un tasto, il mouse può segnalare che la sua posizione è cambiata oppure uno dei suoi tasti è stato premuto, il disco può indicare che l'operazione di scrittura su un certo file è terminata e così via.

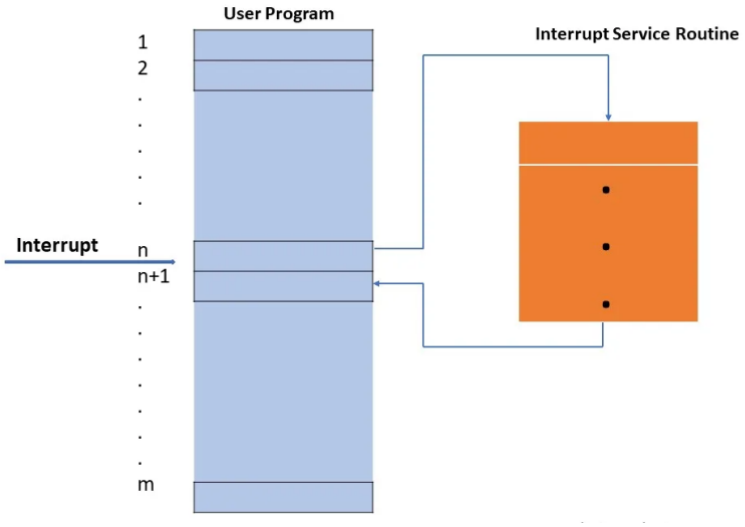
Quando la CPU riceve l'interruzione, viene effettuato il passaggio in Kernel Mode per poter eseguire la routine (funzione) in grado di gestire l'interrupt ricevuta.

Una volta che tale funzione è terminata, la CPU torna a lavorare in User Mode, riprendendo l'esecuzione dal punto in cui era stata sospesa.



Una seconda modalità con la quale può essere fatto il passaggio alla Kernel Mode è su richiesta: quando un software applicativo ha bisogno di richiamare una funzionalità (un servizio) offerta dal sistema operativo, effettuerà una **system call** (chiamata di sistema), ovvero andrà a richiamare una delle funzioni messe a disposizione in una libreria di sistema (**API** – **A**pplication **P**rogramming **I**nterface).

Questa funzione preparerà sullo stack tutti i dati necessari ed invocherà un **interrupt software**, che, esattamente come quella hardware, farà passare la CPU alla Kernel Mode per l'esecuzione della routine dedicata a rispondere a tale interrupt; terminata la routine, la CPU torna in User Mode e prosegue con l'esecuzione del programma dal punto in cui era stata interrotta.



Dal punto di vista del programmatore, non si ha quasi mai la necessità di utilizzare le system call, perché questo richiederebbe la conoscenza di come preparare e richiamare una interrupt software. Quello che tipicamente succede

è che vengano utilizzate delle funzioni classiche che, al loro interno, effettuano in maniera automatica la chiamata alle system call necessarie.